

Linear Search

Check one position at a time and return the first matching index; report -1 only after every position has failed.

How to use this chapter:

Learn the idea in plain language, read each diagram, predict every trace row, then study the commented Java and attempt the exam-style questions.

The whole process at a glance

Check one position at a time and return the first matching index; report -1 only after every position has failed.

Look along the row of cards until the value you want appears. Return the position, not the card's value.

Search for 18. Only the index moves: failed checks advance left to right, and the first match returns immediately.

Green: sorted / found

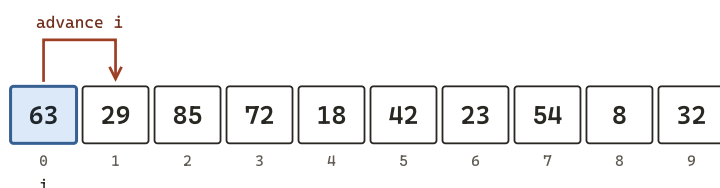
Blue: chosen value / midpoint

Orange: destination / held key

Crossed: ruled out

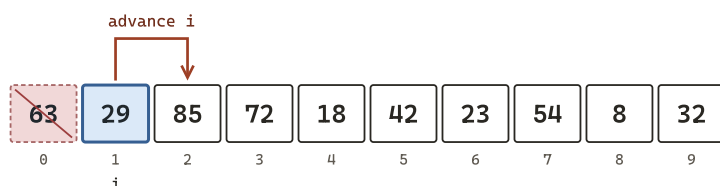
· = conceptual gap

Check 1



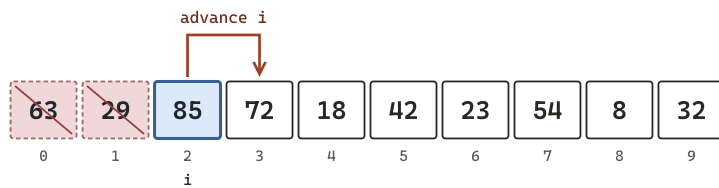
63 == 18 is false. Advance i to 1.

Check 2



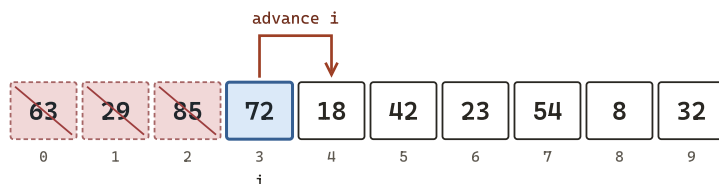
29 == 18 is false. Advance i to 2.

Check 3



85 == 18 is false. Advance i to 3.

Check 4



72 == 18 is false. Advance i to 4.

Check 5



18 == 18: return index 4. Stop.

Return 4 after 5 key comparisons. Later cells are not inspected after the match.

Searching / Linear search · The textbook definition

The textbook definition

Linear search (also called *sequential search*) finds a value in a collection by examining the elements one at a time, in order, starting from the first. It stops as soon as an element equals the target and returns that element's index. If it reaches the end without a match it reports failure, by convention by returning -1 . It works on any array, sorted or not.

Key The value we are looking for. Also called the target or search value.	Index The position of a value in the array, starting at 0.
Probe / comparison One equality test, <code>arr[i] == key</code> . The cost of a search is measured in these.	Match A cell whose value equals the key.
Found The search returns the <i>index</i> of the match.	Not found The search returns <code>-1</code> , a value that can never be a valid index.
Early exit Stopping at the first match instead of examining the rest.	

“Linear” describes the cost: in the worst case you look at every one of the n elements, so the work grows in a straight line with the size of the array.

Searching / Linear search · In plain language

In plain language

Open the drawers one by one until you find what you are looking for, or run out of drawers.

Imagine a row of ten closed drawers and you are looking for the one containing your keys. You have no idea which it is, so you open the first drawer, then the second, then the third, until you find them. If you open the last drawer and still have not found them, they are not in the row.

That is the entire algorithm. There is no clever shortcut because nothing tells you where to look. The only way to be sure a value is *not* there is to have looked everywhere.

Here is the search for 18, one step at a time. Crossed-out cells are ruled out. The labelled cell, marked `i`, is the one being examined.

Step 1: look at index 0: is 63 == 18? No



Step 2: look at index 1: is 29 == 18? No



Step 3: look at index 2: is 85 == 18? No



Step 4: look at index 3: is 72 == 18? No



Step 5: look at index 4: is 18 == 18? Yes, found



It took **5 comparisons**: four misses and one hit, and the search stopped immediately at the hit. The five cells to the right of it were never looked at.

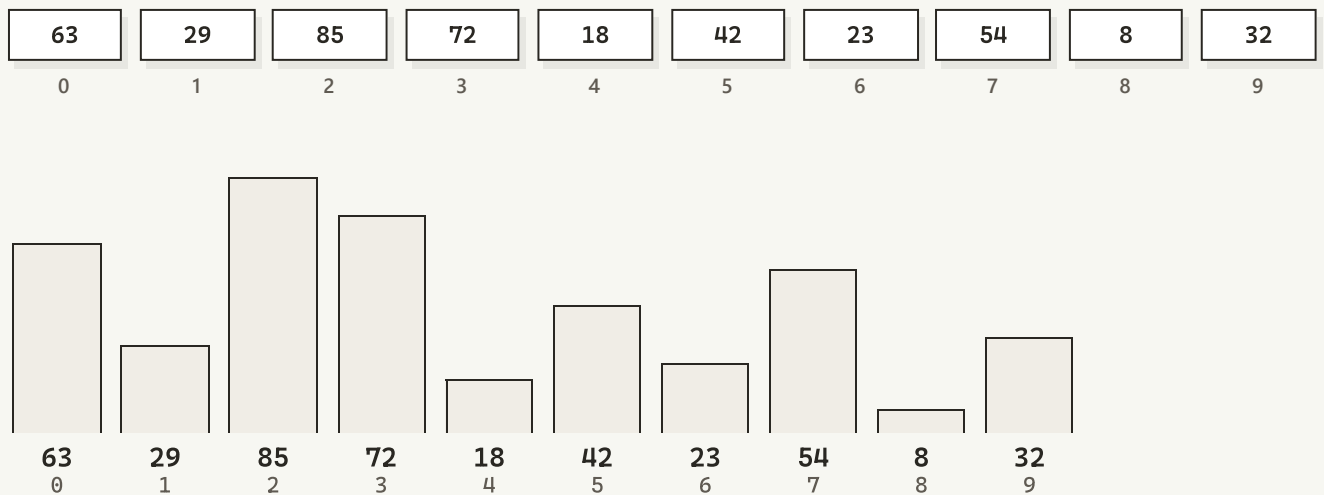
Detailed reference example: this part introduces its own input, shown below. The opening whole-run map and guided walkthrough keep their original input throughout; each later worked example states its new input explicitly.

Searching / Linear search · The array we will search

The array we will search

Ten values in no particular order: [63, 29, 85, 72, 18, 42, 23, 54, 8, 32].

AN UNSORTED ARRAY OF TEN VALUES



Look at the bars: there is no order to exploit. The values jump up and down, so nothing about one cell tells you anything about the next. That is the situation linear search is built for.

We will use two searches throughout this chapter: for 18, which **is** in the array (at index 4), and for 31, which is **not**.

Searching / Linear search · The main idea

The main idea

Linear search is built on one statement that stays true all the way through, called a **loop invariant**:

Invariant: when the loop is about to examine index i , the key is *not* in cells $0 \dots i - 1$.

The loop is about to examine index 0: nothing is ruled out yet



The loop is about to examine index 3: indexes 0..2 are ruled out



The loop is about to examine index 7: indexes 0..6 are ruled out

63	29	85	72	18	42	23	54	8	32
0	1	2	3	4	5	6	7	8	9

Each failed comparison enlarges the ruled-out region by one cell. That has two consequences:

- If a comparison succeeds at index `i`, the key is there, and returning `i` is correct.
- If the loop ends, *every* cell has been ruled out, so returning `-1` is correct. The `return -1` belongs **after** the loop, never inside it.

1

Start at index 0.

2

Compare `arr[i]` with the key.

3

Equal? Return `i`. Not equal? Move to `i + 1`.

4

Past the last index? Return `-1`.

Searching / Linear search · Comparison, match, return

Comparison, match, return

A linear search only ever does two things: compare and return. Being precise about both is what makes a trace correct.

Comparison <code>arr[i] == key</code> . Only equality is tested, never <code><</code> or <code>></code> , which is why the order of the array does not matter.	Match The comparison is true. The search ends here.
Return value The index on success, <code>-1</code> on failure. Some programs return <code>true</code> / <code>false</code> instead; that changes the answer's type, not the work done.	Miss A false comparison. The search moves on.

Why `-1`?

The return value has to tell the caller two things with one number: *where*, or *not found*. Every real index is `0` or more, so `-1` can never be confused with one. Returning `0` for “not found” would be a bug, because index `0` is a real position.

The same array, four searches

KEY	RETURNED	COMPARISONS	WHY
63	0	1	first cell: the best case
18	4	5	found in the middle
32	9	10	last cell: the worst case that succeeds
31	-1	10	not present: the worst case

Notice that the answer is not decided by how big the array is but by **where** the key happens to sit.

Guided walkthrough: predict the next major step

Predict the next completed check, then advance. Each scene keeps all its movements together.

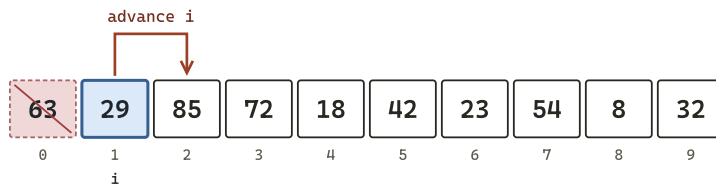
Guided walkthrough: major steps

Check 1

63 == 18 is false. Advance `i` to 1.

63 == 18 is false. Advance i to 1.

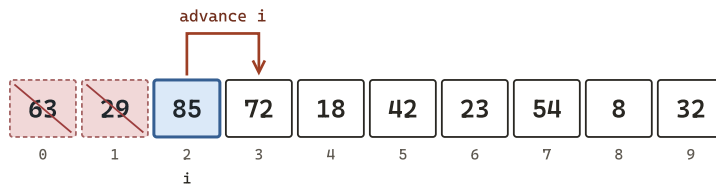
Check 2



29 == 18 is false. Advance i to 2.

29 == 18 is false. Advance i to 2.

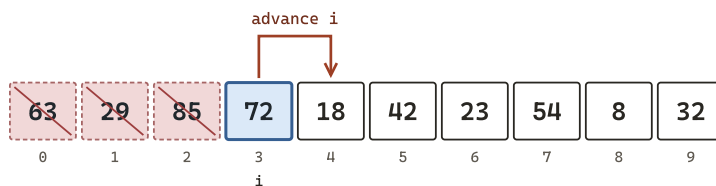
Check 3



85 == 18 is false. Advance i to 3.

85 == 18 is false. Advance i to 3.

Check 4



72 == 18 is false. Advance i to 4.

72 == 18 is false. Advance i to 4.

Check 5



18 == 18: return index 4. Stop.

18 == 18: return index 4. Stop.

▼ Full trace table - select a row to jump to its scene

1	0	63	63 == 18 is false. Advance i to 1.
2	1	29	29 == 18 is false. Advance i to 2.
3	2	85	85 == 18 is false. Advance i to 3.
4	3	72	72 == 18 is false. Advance i to 4.
	4	18	18 == 18: return index 4. Stop.
CHECK	INDEX	VALUE	DECISION

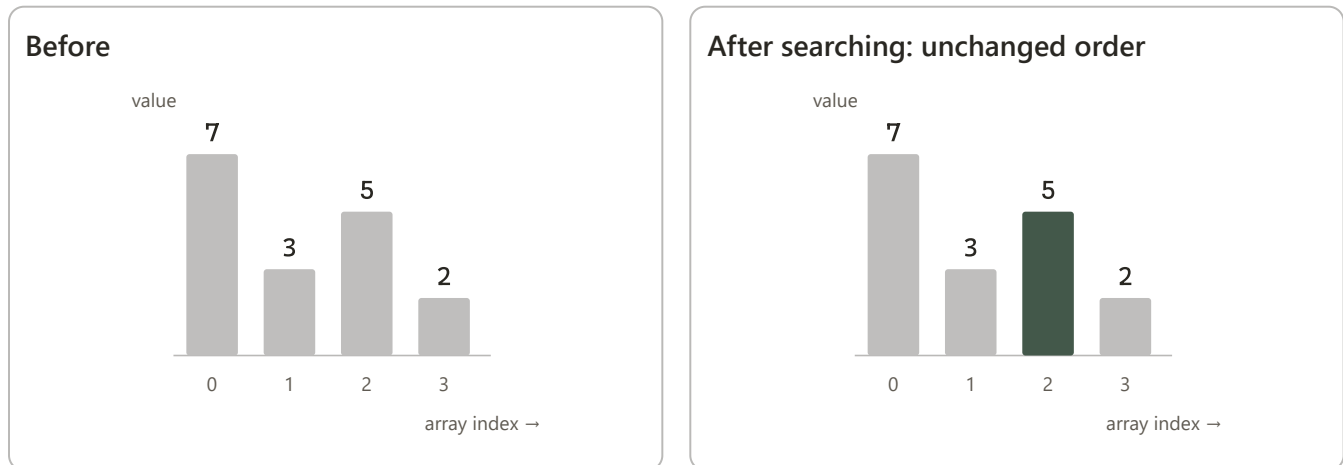
The trace at a glance

CHECK	INDEX	VALUE	DECISION
1	0	63	63 == 18 is false. Advance i to 1.
2	1	29	29 == 18 is false. Advance i to 2.
3	2	85	85 == 18 is false. Advance i to 3.
4	3	72	72 == 18 is false. Advance i to 4.
5	4	18	18 == 18: return index 4. Stop.

Read the algorithm as a bar graph

Bar height means **value**; horizontal position means **index**. All panels below use the same height scale. A taller bar is not “more sorted”.

Searching visits bars; it does not rearrange them



Searching for 5 checks indexes 0, 1, then 2 and returns 2. Heights and positions are unchanged. The selected bar marks where the key was found, not a sorted position.

Check yourself: identify which indexes are guaranteed final, which values were merely inspected, and whether the operation swapped, shifted or copied data. A sorted ascending array has non-decreasing heights, including equal-height neighbours.

Searching / Linear search

Check one position at a time

Linear search—also called sequential search—starts at index 0 and compares the key with each value until it finds a match or reaches the end.

Illustration

Search for 18 from left to right in the unsorted lecture array. The highlighted cell is where the search stops.



Trace table

COMPARISON	INDEX	QUESTION	DECISION
1	0	63 == 18	No · continue
2	1	29 == 18	No · continue
3	2	85 == 18	No · continue
4	3	72 == 18	No · continue
5	4	18 == 18	Yes · return 4

Implementation

Iterative linear search

```
public static int linearSearch(int[] arr, int key) {  
    // Visit every valid index from left to right.  
    for (int i = 0; i < arr.length; i++) {  
        // Stop immediately when the current value matches.  
        if (arr[i] == key) return i;  
    }  
    // Reaching this line means that no position matched.  
    return -1;  
}
```

▼ Recursive implementation

Java

```
public static int linearSearchRec(int[] arr, int key, int index) {  
    // Base case: index moved beyond the final position.  
    if (index == arr.length) return -1;  
    // Success case: the current position contains the key.  
    if (arr[index] == key) return index;  
    // Smaller problem: search from the next index.  
    return linearSearchRec(arr, key, index + 1);  
}
```

Write it from scratch

- 1

Accept the array and key.
- 2

Loop from index 0 to `length - 1`.
- 3

Return the matching index.
- 4

After the loop, return `-1`.

Ascending or descending?

Linear search uses equality only, so the same code works for ascending, descending, or unsorted arrays.

Debug checklist

- Use `i < arr.length`, not `≤`.
- Put `return -1` after the loop.
- Start recursion with index 0.

Worked example · key absent

Search for 31 in the same unsorted array. Every cell is crossed out because each one failed its equality test. The label under a cell shows its index and the order in which it was checked.

Complete scan

63	29	85	72	18	42	23	54	8	32
0 · 1ST	1 · 2ND	2 · 3RD	3 · 4TH	4 · 5TH	5 · 6TH	6 · 7TH	7 · 8TH	8 · 9TH	9 · 10TH

No value equals 31 → return -1 after 10 comparisons.

INDEXES TESTED	VALUES TESTED	MATCH?	RETURN
0 → 9	63, 29, 85, 72, 18, 42, 23, 54, 8, 32	No	-1

BEST O(1)	AVERAGE O(n)
WORST O(n)	SORTED? Not needed
ITERATIVE SPACE O(1)	RECURSIVE SPACE O(n)
DUPLICATES First match	INPUT CHANGED? No

Duplicates, Strings, and objects

Duplicates. The loop returns at the *first* match. In [7, 3, 7, 1] searching for 7 returns 0; index 2 is never examined.

7	3	7	1
0	1	2	3

Strings and objects. For primitive cells == compares values. For reference cells it compares references, so text must be compared with .equals.

Search a String array

```

public static int linearSearchStrings(String[] names, String key) {
    for (int i = 0; i < names.length; i++) {
        // Unfilled cells are null; calling equals on null would throw.
        // equals compares the text. == would only compare references.
        if (names[i] != null && names[i].equals(key)) return i;
    }
    return -1;
}

```

CALL	ARRAY	RESULT	REASON
linearSearchStrings(names, "Raj")	{"Ana", null, "Raj"}	2	Index 1 is skipped safely because it is null.
linearSearchStrings(names, "Zed")	same	-1	All three positions were checked.

▼ Quick check: why `names[i] != null` first?

A cell that was never assigned holds `null`. Calling `.equals` through it throws `NullPointerException`. The `&&` stops before that call.

▼ Quick check: can `new String("Raj") == "Raj"` be true?

No. `new String( ... )` creates a different object, so the references differ even though the text matches. `.equals` returns true.

Questions

▼ When is linear search a sensible choice?

When the collection is small, unsorted, or searched only once. Sorting first would add unnecessary work.

▼ What are the best and worst cases?

Best: the key is at index 0, so one comparison gives $O(1)$. Worst: the key is last or absent, so all n values are checked: $O(n)$.

Linear search in one sentence

Check each value in order; return its index on a match, otherwise return `-1` after the final position.

Fully annotated Java: connect each line to the movement

java

```
public static int linearSearch(int[] arr, int key) {  
    // Every valid index must be checked until a match is found.  
    for (int i = 0; i < arr.length; i++) {  
        // Return the INDEX, not the value stored there.  
        if (arr[i] == key) return i;  
    }  
    // Only report failure after all valid indexes were checked.  
    return -1;  
}
```

Build it from scratch

- Accept an array and key.
 - Visit every valid index from 0.
 - Return the matching index immediately.
 - Put return -1 after the loop.
-

Recursion, calls and returns

java

```
public static int linearSearchRec(int[] arr, int key, int index) {  
    // Empty remaining suffix: do not access arr[index].  
    if (index ≥ arr.length) return -1;  
    // Equality finds the first matching position from this index.  
    if (arr[index] == key) return index;  
    // Start the first call at index 0; each call shrinks the suffix.  
    return linearSearchRec(arr, key, index + 1);  
}
```

CALL STACK - NEWEST FRAME AT THE TOP

linearSearchRec(index 2)

Check the current suffix

linearSearchRec(index 1)

Waiting: Return child result

linearSearchRec(index 0)

Waiting: Return child result

Each failure suspends a parent and searches a smaller suffix. Ordinary Java does not eliminate this recursion stack.

Initial call: `linearSearchRec(arr, key, 0)`. Worst-case depth is $O(n)$; the iterative version is preferable when that stack cost matters.

A complete runnable Java example

Save as `LinearDemo.java`, compile with `javac`, then run the class name without an extension.

java

```
import java.util.Arrays;

public class LinearDemo {
    public static int linearSearch(int[] arr, int key) {
        // Every valid index must be checked until a match is found.
        for (int i = 0; i < arr.length; i++) {
            // Return the INDEX, not the value stored there.
            if (arr[i] == key) return i;
        }
        // Only report failure after all valid indexes were checked.
        return -1;
    }

    public static void main(String[] args) {
        int[] values = {63, 29, 85, 72, 18, 42, 23, 54, 8, 32};
        System.out.println(linearSearch(values, 18));
    }
}
```

Expected output

4

Searching / Linear search · Properties

Properties

BEST TIME

$O(1)$

AVERAGE TIME

$O(n)$

WORST TIME

$O(n)$

EXTRA SPACE

$O(1)$

NEEDS SORTED DATA

No

WORKS ON

Any array or list

COMPARES WITH

`==` / equals

FINDS

First match

No precondition

It needs the data in no particular order, and it does not care whether the array is ascending, descending or random. This is its great strength: it always works.

Simple and safe

There is no arithmetic on indexes, nothing to overflow, and no way to skip over the answer. It is hard to get wrong.

Any structure you can walk

It needs only “give me the next element”, so it works on a linked list, a file read line by line, or any stream, where jumping to the middle (as binary search does) is impossible.

The price is time

The cost grows in a straight line with the number of values. Search a million values and a miss costs a million comparisons.

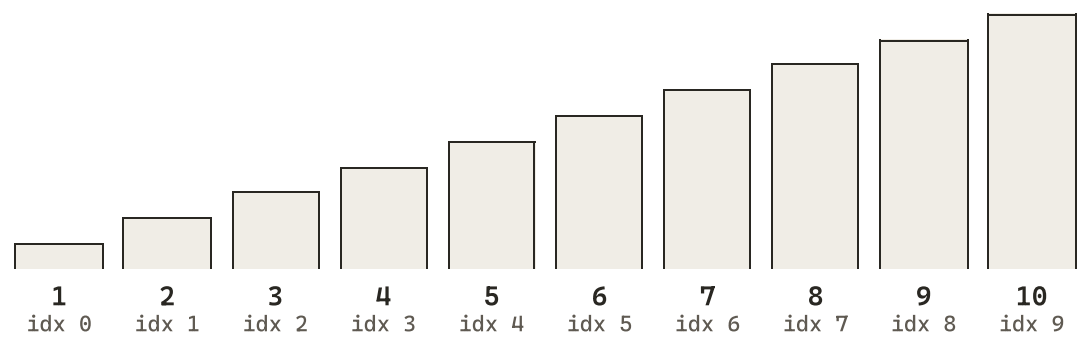
Searching / Linear search · Why those complexities occur

Why those complexities occur

Best case · O(1) The key is in the first cell: one comparison, whatever the size.	Worst case · O(n) The key is in the last cell or missing: all n comparisons.
---	--

The cost depends on where the key is

For our ten values, the number of comparisons needed to find a key is its index plus one. Here is that cost for a key in each position:



A taller bar means more comparisons. A key that is absent costs the full height of the last bar, 10.

The average case

If the key is equally likely to be in any cell, the average cost is the middle of 1 and 10:

$$(1 + 2 + \dots + n) / n = (n + 1) / 2 \rightarrow O(n)$$

For ten values that is 5.5 comparisons. Halving the cost still leaves a cost proportional to n, so the order of growth is linear.

N (VALUES)	WORST CASE	AVERAGE (KEY PRESENT)	ABSENT KEY
10	10	5.5	10
100	100	50.5	100
1,000	1,000	500.5	1,000
1,000,000	1,000,000	500,000.5	1,000,000

The pattern to remember: **ten times as many values means ten times as many comparisons**. Compare that with the next chapter, where ten times as many values adds only about 3 more comparisons.

Space

An index and the key: $O(1)$ extra space (the recursive version uses $O(n)$ stack space).

Searching / Linear search · When to use it

When to use it

A good choice The data is not sorted and is searched only once or twice; small collections (a handful of values); a quick check inside a bigger program; data in a structure with no random access (linked list, file); when you need the first match, or all matches.	A poor choice A large array searched many times (sort it once, then use binary search); when the data is already sorted (binary search is far faster); when speed matters and you can organise the data in advance.
---	---

The honest comparison: sorting an array first costs about $n \log n$. If you will only search once, scanning (n) is cheaper than sorting and then binary searching. If you will search many times, the one-off sort pays for itself, as the binary-search chapter shows with numbers.

Searching

What searching means

Searching is the process of looking for a particular element in a collection. A successful method returns its index; an unsuccessful search returns -1 .

Search key The value being looked for.	Comparison Checking the key against an array value.
Ordered collection Values already arranged by a rule.	Unordered collection Values may appear in any order.

Use a search to check membership, display or update an item, or find the position needed for another operation.

Reading Big-O: why we count steps

Big-O describes how the number of steps grows as the number of values n grows. It ignores small constant details and keeps the dominant growth. *Best*, *average* and *worst* case describe three situations for the same size n : the most favourable input, a typical input, and the least favourable input. In this guide one *comparison* means one probe: one look at an array value (for binary search, `arr[mid]`), even though Java may spell it as two tests such as `==` then `<`.

N VALUES	BINARY SEARCH $O(\log N)$ WORST COMPARISONS	LINEAR SEARCH $O(N)$	MERGE SORT $O(N \log N)$	SIMPLE SORTS $O(N^2)$ WORST
10	4	10	≈ 33	100
100	7	100	≈ 664	10,000
1,000	10	1,000	$\approx 9,966$	1,000,000
1,000,000	20	1,000,000	≈ 20 million	1,000,000,000,000

Binary search column: $\lfloor \log_2 n \rfloor + 1$ comparisons. Merge column: $n \times \log_2 n$. These are step counts, not seconds.

$O(1)$ · constant The work does not depend on n , such as reading <code>marks[0]</code> .	$O(\log n)$ · logarithmic Each step discards a fraction of the data. Doubling n adds one step.
$O(n)$ · linear Each value is touched about once. Doubling n doubles the work.	$O(n^2)$ · quadratic Each value may meet every other value. Doubling n quadruples the work.

Searching / Linear search · Variations: first, last, all, count

Variations: first, last, all, count

When the key appears more than once, “find it” is ambiguous. Linear search handles every version with a small change, and they are all the same walk along the array. Take `[7, 3, 7, 1, 7]` and the key 7:

7	3	7	1	7
0	1	2	3	4

QUESTION	HOW	ANSWER	COMPARISONS
First match	Return as soon as you find it (the standard version).	index 0	1
Last match	Do not stop: remember the latest index, or search from the right.	index 4	5
Every match	Do not stop: collect each matching index.	indexes [0, 2, 4]	5
How many	Do not stop: add 1 to a counter for each match.	3 times	5

First match (stops early)

```
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == key) return i;
}
return -1;
```

Last match (never stops early)

```
int found = -1;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == key) found = i;
}
return found;
```

Count every occurrence

```
int count = 0;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == key) count++;
}
return count;
```

The first-match version is the only one that can finish early. Any question that needs *all* the matches, or the last, forces a full scan every time: n comparisons whether or not the key is present.

Searching / Linear search · Worked examples

Worked examples

Six searches: best case, typical case, worst case that succeeds, failure, Strings, and the unsorted array from the past paper.

Key in the first cell · search 63

The luckiest possible search.

63	29	85	72	18	42	23	54	8	32
0	1	2	3	4	5	6	7	8	9

COMPARISON	INDEX	QUESTION	DECISION
1	0	63 == 63	Yes: return 0

1 comparison, returns 0. This is the best case: $O(1)$, no matter how long the array is.

Key in the middle · search 18

The ordinary case.

63	29	85	72	18	42	23	54	8	32
0	1	2	3	4	5	6	7	8	9

COMPARISON	INDEX	QUESTION	DECISION
1	0	63 == 18	No: continue
2	1	29 == 18	No: continue
3	2	85 == 18	No: continue
4	3	72 == 18	No: continue
5	4	18 == 18	Yes: return 4

5 comparisons, returns 4. The cost is the key's position plus one.

Key in the last cell · search 32

The unluckiest search that still succeeds.

63	29	85	72	18	42	23	54	8	32
0	1	2	3	4	5	6	7	8	9

COMPARISON	INDEX	QUESTION	DECISION
1	0	63 == 32	No: continue
2	1	29 == 32	No: continue
3	2	85 == 32	No: continue
4	3	72 == 32	No: continue
5	4	18 == 32	No: continue
6	5	42 == 32	No: continue
7	6	23 == 32	No: continue
8	7	54 == 32	No: continue
9	8	8 == 32	No: continue
10	9	32 == 32	Yes: return 9

10 comparisons: all $n = 10$ of them. A worst case: $O(n)$.

Key not present · search 31

Failure is the most expensive outcome, because only a full scan can prove absence.

63	29	85	72	18	42	23	54	8	32
0	1	2	3	4	5	6	7	8	9

COMPARISON	INDEX	QUESTION	DECISION
1	0	63 == 31	No: continue
2	1	29 == 31	No: continue
3	2	85 == 31	No: continue
4	3	72 == 31	No: continue

COMPARISON	INDEX	QUESTION	DECISION
5	4	18 == 31	No: continue
6	5	42 == 31	No: continue
7	6	23 == 31	No: continue
8	7	54 == 31	No: continue
9	8	8 == 31	No: continue
10	9	32 == 31	No: continue
end of the array			return -1

10 comparisons and returns -1. Also the worst case: $O(n)$.

Searching Strings · find "Lee" in [Raj, Ana, Lee, Mike, Zed]

Same algorithm, but the comparison must be `.equals`, not `==`.

Raj	Ana	Lee	Mike	Zed
0	1	2	3	4

COMPARISON	INDEX	QUESTION	DECISION
1	0	Raj == Lee	No: continue
2	1	Ana == Lee	No: continue
3	2	Lee == Lee	Yes: return 2

3 comparisons, returns 2. For Strings write `arr[i].equals(key)`: `==` compares references, not letters.

The nine values from the past paper · search 88

Unsorted, and 88 is not in it. Binary search cannot be used here, because the array is not sorted.

50	83	76	6	32	90	12	2	59
0	1	2	3	4	5	6	7	8

COMPARISON	INDEX	QUESTION	DECISION
1	0	50 == 88	No: continue
2	1	83 == 88	No: continue
3	2	76 == 88	No: continue
4	3	6 == 88	No: continue
5	4	32 == 88	No: continue
6	5	90 == 88	No: continue
7	6	12 == 88	No: continue
8	7	2 == 88	No: continue
9	8	59 == 88	No: continue
end of the array			return -1

9 comparisons and -1: every cell had to be checked. If the array were sorted, binary search would settle the same question in 4 steps (see the binary-search chapter).

Searching / Linear search · Common mistakes

Common mistakes

1 Returning -1 inside the loop.

`for ( ... ) { if (arr[i] == key) return i; else return -1; }` gives up after the *first* cell. The failure return belongs after the loop.

2 **Looping to $i \leq \text{arr.length}$.**
Index `arr.length` does not exist and throws `ArrayIndexOutOfBoundsException`. Use `<`.

3 **Returning 0 for “not found”.**
0 is a valid index. Use `-1`.

4 **Comparing Strings with `==`.**
That compares references. Use `arr[i].equals(key)`; and test for `null` cells first if the array may have empty slots.

5 **Assuming the array must be sorted.**
Linear search does not need sorting. Sorting first just to linear search wastes time.

6 **Counting a comparison after the match.**
The search stops at the match, so the comparison that succeeds is the last one counted.

7 **Returning the value instead of the index.**
The caller already knows the value (it is the key). The useful information is the position.

8 **Believing it is “slow”.**
It is only slow on large data searched repeatedly. For ten values it is as fast as anything.

Searching / Linear search · A test-ready summary

A test-ready summary

TEST ANSWER

Linear search examines the elements of an array one at a time from the first, comparing each with the key, and returns the index of the first match or -1 if the end is reached without one. It needs no sorted data and works on any collection. Its best case is $O(1)$ (key in the first cell), and its average and worst cases are $O(n)$ (about $n / 2$ and n comparisons), with $O(1)$ extra space. It is the right choice for small or unsorted data searched rarely, and the wrong choice for large collections searched repeatedly, where sorting once and using binary search is faster.

Searching / Linear search · Practice

Practice

Guided question

Use linear search to find 7 in [4, 9, 7, 2, 7]. Write each comparison, say what is returned, and count the comparisons.

▼ Reveal the guided answer

Comparison 1: 4 == 7 → no

Comparison 2: 9 == 7 → no

Comparison 3: 7 == 7 → yes, return 2

Result: returns 2 after 3 comparisons. The second 7 (index 4) is never examined, because the search stops at the first match.

Independent question

Search for 12 in [8, 3, 9, 5, 12, 1], then for 4. Give the index returned and the number of comparisons for each.

Write the comparison for each index.

- ⊗ Stop at the first match.
- If there is no match, say why the answer is -1 .

▼ **Reveal the independent answer**

Key 12: returns **4** after **5** comparisons.
Key 4: returns **-1** after **6** comparisons, because every cell had to be checked to prove 4 is absent.

Test-style question

How many comparisons does linear search need to decide that 88 is not in [50, 83, 76, 6, 32, 90, 12, 2, 59]? What is the worst-case time complexity in big-O notation?

▼ **Reveal the test-style answer**

9 comparisons (one for every value), returning -1. Worst case: **$O(n)$** .

More worked cases: edge cases, duplicates and descending order

1. Absent key

Input [63, 29, 85, 72, 18, 42, 23, 54, 8, 32]; key 31.

63

29

85

72

18

42

23

54

8

012345678

32

9

Final probe/check at index 9; return -1.

CHECK	i	ARR[i]	EVALUATED CONDITION AND RESULT
1	0	63	63 == 31 → false; advance i

CHECK	I	ARR[I]	EVALUATED CONDITION AND RESULT
2	1	29	29 == 31 → false; advance i
3	2	85	85 == 31 → false; advance i
4	3	72	72 == 31 → false; advance i
5	4	18	18 == 31 → false; advance i
6	5	42	42 == 31 → false; advance i
7	6	23	23 == 31 → false; advance i
8	7	54	54 == 31 → false; advance i
9	8	8	8 == 31 → false; advance i
10	9	32	32 == 31 → false; advance i

Result -1; 10 key comparisons.

2. First match among duplicates

Input [7, 3, 7, 1]; key 7.

7

3

7

1

0123

Final probe/check at index 0; return 0.

CHECK	I	ARR[I]	EVALUATED CONDITION AND RESULT
1	0	7	7 == 7 → true; return 0

Result 0; 1 key comparisons.

3. Match at the last index

Input [9, 4, 2, 18]; key 18.

9	4	2	18
0	1	2	3

Final probe/check at index 3; return 3.

CHECK	I	ARR[I]	EVALUATED CONDITION AND RESULT
1	0	9	$9 == 18 \rightarrow \text{false}$; advance i
2	1	4	$4 == 18 \rightarrow \text{false}$; advance i
3	2	2	$2 == 18 \rightarrow \text{false}$; advance i
4	3	18	$18 == 18 \rightarrow \text{true}$; return 3

Result 3; 4 key comparisons.

4. Negative values

Input [-5, -2, 0, 8]; key -2.

-5	-2	0	8
0	1	2	3

Final probe/check at index 1; return 1.

CHECK	I	ARR[I]	EVALUATED CONDITION AND RESULT
1	0	-5	$-5 == -2 \rightarrow \text{false}$; advance i
2	1	-2	$-2 == -2 \rightarrow \text{true}$; return 1

Result 1; 2 key comparisons.

5. Empty and singleton

Input [18]; key 31.

18

0

Final probe/check at index 0; return -1.

CHECK	I	ARR[I]	EVALUATED CONDITION AND RESULT
1	0	18	18 == 31 → false; advance i

Result -1; 1 key comparisons. For [] the loop/body is never entered and -1 is returned with zero probes.

Exam-style practice with worked answers

▼ A. Trace key 31 on the main input. Show every index/bound and the failure result. [6]

CHECK	I	ARR[I]	EVALUATED CONDITION AND RESULT
1	0	63	$63 == 31 \rightarrow \text{false}$; advance i
2	1	29	$29 == 31 \rightarrow \text{false}$; advance i
3	2	85	$85 == 31 \rightarrow \text{false}$; advance i
4	3	72	$72 == 31 \rightarrow \text{false}$; advance i
5	4	18	$18 == 31 \rightarrow \text{false}$; advance i
6	5	42	$42 == 31 \rightarrow \text{false}$; advance i
7	6	23	$23 == 31 \rightarrow \text{false}$; advance i
8	7	54	$54 == 31 \rightarrow \text{false}$; advance i
9	8	8	$8 == 31 \rightarrow \text{false}$; advance i
10	9	32	$32 == 31 \rightarrow \text{false}$; advance i

Every retained candidate is eliminated; return -1. Ten equality checks are required.

▼ B. Explain the best and worst time using input examples. [4]

Best: key at index 0, one equality test. Worst: key last or absent, n equality tests.

▼ C. Debug the bounds and return statement. [4]

i must stay below length; a failed equality test means continue, not return -1. Failure belongs after the loop.

▼ D. Return the first occurrence of a duplicate. What must change? [3]

Nothing: the left-to-right immediate return already selects the first occurrence.

▼ E. Empty array: can this code access index 0? [2]

No. $i = 0$ is not less than $\text{length} = 0$. It returns -1 without reading a cell.

Topic quiz: test your understanding

Answer all 12 questions. Choose an option to see the reasoning; reset to retry. The printable answer key follows the questions.

1. What is the last valid index of an array of length 6?

- ☐ A. 5
- ☐ B. 6
- ☐ C. 7

2. What does $O(1)$ auxiliary space describe?

- ☐ A. Exactly one machine instruction
- ☐ B. The input contains one value
- ☐ C. Extra storage stays bounded as n grows

3. What is a loop invariant?

- ☐ A. A loop that never terminates
- ☐ B. A statement that remains true at a specified loop boundary
- ☐ C. A variable that never changes

4. Does linear search require sorted input?

- ☐ A. No
- ☐ B. Yes, ascending only
- ☐ C. Yes, descending only

5. Search [7, 3, 7, 1] for 7 with a first-match scan. Return?

- ☐ A. 2
- ☐ B. 7
- ☐ C. 0

6. When may the method return -1?

- ☐ A. When index 0 is checked
- ☐ B. After every candidate has failed
- ☐ C. After the first failed comparison

7. Worst-case key comparisons for n values?

- ☐ A. n
- ☐ B. $\log_2 n$
- ☐ C. n^2

8. Best-case time?

- ☐ A. $O(n)$
- ☐ B. $O(n^2)$
- ☐ C. $O(1)$

9. What changes during a read-only linear search?

- ☐ A. The length of the array
- ☐ B. The current index
- ☐ C. The order of values

10. What loop bound avoids reading one past the end?

- ☐ A. $i < \text{arr.length}$
- ☐ B. $i \leq \text{arr.length}$
- ☐ C. $i > \text{arr.length}$

11. How should Java String content be compared?

- ☐ A. Always use `==`
- ☐ B. Subtract the two Strings
- ☐ C. Use equals with an appropriate null check

12. What extra space can the recursive scan require?

- ☐ A. $O(n^2)$ array storage
- ☐ B. $O(n)$ call stack
- ☐ C. Always $O(1)$

▼ Answer key and explanations

1. **5.** Indexes start at 0, so the last index is $\text{length} - 1$.
2. **Extra storage stays bounded as n grows.** Auxiliary space counts extra working storage, excluding the input array.
3. **A statement that remains true at a specified loop boundary.** The invariant explains why each iteration preserves progress toward a correct result.
4. **No.** Each position is checked directly; no ordering assumption is used to skip values.
5. **0.** The first comparison matches and returns its index, not its value.
6. **After every candidate has failed.** One failure rules out only one cell. Later cells may still contain the key.
7. **n .** An absent key, or one first found at the last cell, requires a complete scan.
8. **$O(1)$.** The first cell may match regardless of the total input size.
9. **The current index.** Searching inspects values without sorting or moving them.
10. **$i < \text{arr.length}$.** `arr[arr.length]` is outside the array.
11. **Use equals with an appropriate null check.** `==` compares references; `equals` compares String contents.
12. **$O(n)$ call stack.** Each failed recursive call may remain suspended until the final result returns.